Week 9 · Making it look right: visual design & accessibility

Technical Interaction Design · ITU · Autumn 2026

Built 15 October 2026 · commit 6ac857a
Latest version: https://itu-tid.github.io/lecture-notes-pdf/Week-09.pdf

CONTENTS

Libraries

- Most of the code your app runs, you did not write
- `npm` reads `package.json` and fetches everything it lists
- The libraries land in `node_modules`, which never goes into Git
- A version number says how much changed: major, minor, patch
- `^` lets npm take newer minor and patch versions, but not a new major one
- `package-lock.json` pins the whole tree, and you commit it
- Every dependency is a decision someone has to defend

Libraries

Source: Lectures/Technical/Tooling/Libraries.md — something unclear? Open an issue, or send a pull request.

“If I have seen further, it is by standing on the shoulders of giants.” (Isaac Newton)

(the programmers’ version of the same idea)

Most of the code your app runs, you did not write

Look at the `dependencies` in `todo-26`’s `package.json`:

```
"dependencies": {
  "events": "^3.3.0",
  "parse": "^8.6.0",
  "react": "^19.2.8",
  "react-dom": "^19.2.8",
  "react-router-dom": "^7.18.4"
}
```

React itself is on that list. It is a library like any other: you did not write the component model, the router or the Parse SDK, and you should not. Every modern language lets a project declare the libraries it depends on, in one file, and JavaScript’s is `package.json`.

Something unclear in this section? Open an issue.

`npm` reads `package.json` and fetches everything it lists

Each language has its own **package manager**; JavaScript’s is `npm`. It runs on **Node**, which is JavaScript outside the browser, built on the same engine as Chrome (V8). Node is also what runs Vite, the dev server, and the build. Your React app itself still runs in the browser; Node is the workshop it is built in.

```
npm install                    # everything package.json lists
npm install @mui/material      # add one more, and write it into package.json
```

Run `npm install` after you clone a project, and again after every `git pull` that changed `package.json`. If the app suddenly cannot find a package a teammate just added, that is why.

Something unclear in this section? Open an issue.

The libraries land in `node_modules`, which never goes into Git

`npm install` puts the code of every library, and of every library *they* depend on, into `node_modules/`. In `todo-26` that is over two hundred packages and about 120 megabytes, for five lines in `dependencies`.

None of it belongs in the repo. Anybody can recreate it with `npm install`, and some of it is built for one particular operating system. That is why `node_modules` is in `.gitignore`.

Something unclear in this section? Open an issue.

A version number says how much changed: major, minor, patch

Most JavaScript libraries use semantic versioning. In `19.2.8`:

- **19**, the *major* version, changes when something that used to work stops working
- **2**, the *minor* version, changes when something new is added, and the old still works
- **8**, the *patch* version, changes for bug fixes only

Not everybody follows it. Some projects use the date (*calendar versioning*: `pip` 24.0 came out in 2024). TeX's version number gets one more digit of π with every release.

Something unclear in this section? Open an issue.

`^` lets npm take newer minor and patch versions, but not a new major one

`"react": "^19.2.8"` means *19.2.8 or anything newer that is still 19*. That is a **flexible** version: you get bug fixes without asking. `"react": "19.2.8"`, with no `^`, is **pinned**: exactly that version, and nothing else.

But even pinning everything in `package.json` does not pin what your libraries depend on. `parse` has dependencies of its own, which have their own, and their versions are flexible too.

Something unclear in this section? Open an issue.

`package-lock.json` pins the whole tree, and you commit it

So `npm` writes down every version it actually installed, for every package in `node_modules`, in `package-lock.json`. The next `npm install`, on any laptop, installs exactly those.

That is the point of it, and why it goes into Git, even though `npm` generates it. Without it, two people in the same group, who ran `npm install` a week apart, can have different versions of the same library, and a bug that only one of them sees. `todo-26` commits it.

Do not edit it by hand. When two branches both changed dependencies and it conflicts on merge, take either side and run `npm install` again: `npm` will rewrite it.

Something unclear in this section? Open an issue.

Every dependency is a decision someone has to defend

Adding a package takes one command. Getting rid of it, once forty files import it, takes a week. So before you `npm install` something, ask:

- **Do we need it?** Formatting one date does not need a date library.
- **Is it alive?** When was the last release? Does anybody answer its issues?
- **What does it cost?** How much bigger does the app get, and what do you give up by working its way instead of yours?

`react-router-dom` earned its place in week 6: writing your own router is a project of its own. MUI, next, is a bigger decision: a whole design language, in exchange for a type scale, a palette and spacing that already work together. It is the *No needless dependencies* item on the review checklist.

Something unclear in this section? Open an issue.

Exam Questions

1. How do you install a dependency with `npm`, and what do you do to make sure that your teammates use the same version you do?

2. When do you have to run `npm install`?

3. What is the `node_modules` folder for? Should you add it to version control? Why or why not?

4. What do the three numbers in a version like `19.2.8` mean, and what does the `^` in front of it allow?

5. Why do we need `package-lock.json`, and do we commit it?